

Practical Work 5: The Longest Path

Tran Tuan Vu - 23BI14459

December 11, 2025

1 Implementation Choice

For this practical work, we continued using our custom ****C + Pthreads**** MapReduce framework.

- **Framework:** Custom C implementation using POSIX threads.
- **Input Strategy:** Each input file is treated as a distinct data partition (split) assigned to a single Mapper thread. This simulates a real-world scenario where logs from different machines (e.g., "laptop1.log", "laptop2.log") are processed in parallel.

2 System Design

The problem of finding the longest path is an aggregation problem that fits MapReduce perfectly.

1. **Map Phase:** Instead of emitting every single line (which would be inefficient), each Mapper processes its entire assigned file and finds the *local maximum* (the longest path within that specific file). It emits a single key-value pair: `<"MAX", local_longest_path>`.
2. **Shuffle Phase:** The master thread collects the `MapperResult` structures from all threads into a central array.
3. **Reduce Phase:** The Reducer iterates through the array of local maximums to find the *global maximum*.

3 Implementation Details

3.1 The Mapper

The mapper reads lines, strips newlines, and compares lengths locally.

```
1 void *mapper(void *arg) {
2     // ... File opening logic ...
3     while (fgets(line, sizeof(line), fp)) {
4         int len = strlen(line);
5         if (len > args->result->length) {
6             args->result->length = len;
7             strcpy(args->result->longest_path, line);
8         }
9     }
10    // Only the single longest path is "emitted" to the result struct
```

```

11     return NULL;
12 }

```

Listing 1: Mapper Local Search

3.2 The Reducer

The reducer is very lightweight; it simply performs a maximum value search on the results provided by the mappers.

```

1 void reducer(MapperResult *results, int count) {
2     int max_len = -1;
3     // Iterate through all mapper outputs
4     for (int i = 0; i < count; i++) {
5         if (results[i].length > max_len) {
6             max_len = results[i].length;
7             strcpy(global_longest, results[i].longest_path);
8         }
9     }
10    printf("Final Longest Path: %s\n", global_longest);
11 }

```

Listing 2: Reducer Aggregation

4 Experimental Results

We tested the system using two input files simulating file system logs from different laptops.

```

1 $ ./longest_path laptop1.txt laptop2.txt
2 [Master] Spawning 2 Mapper threads...
3 [Mapper] Finished laptop2.txt. Longest here: 10 chars.
4 [Mapper] Finished laptop1.txt. Longest here: 58 chars.
5
6 [Reducer] Aggregating results from 2 mappers...
7 -----
8 FINAL RESULT: The Longest Path
9 -----
10 Length: 58 characters
11 Path:    /var/log/system/very_long_path_name_for_testing_purposes.log

```

Execution Log